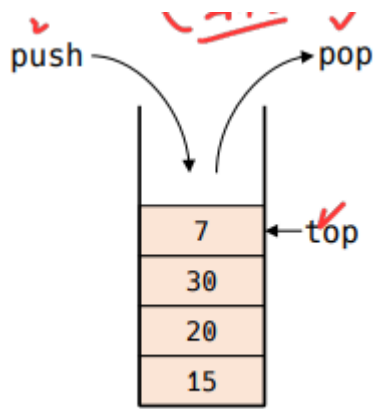


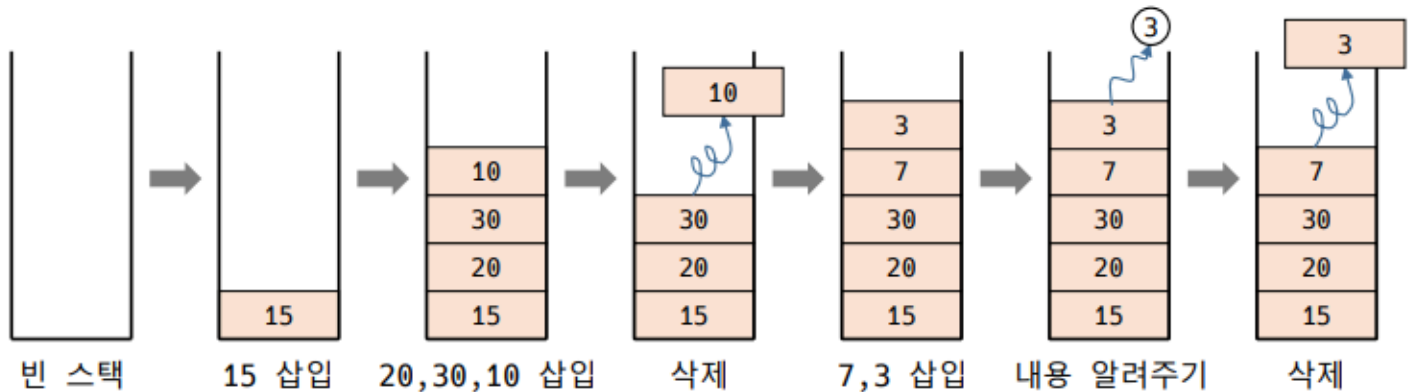
Chap 6 스택 정리

0. 개념

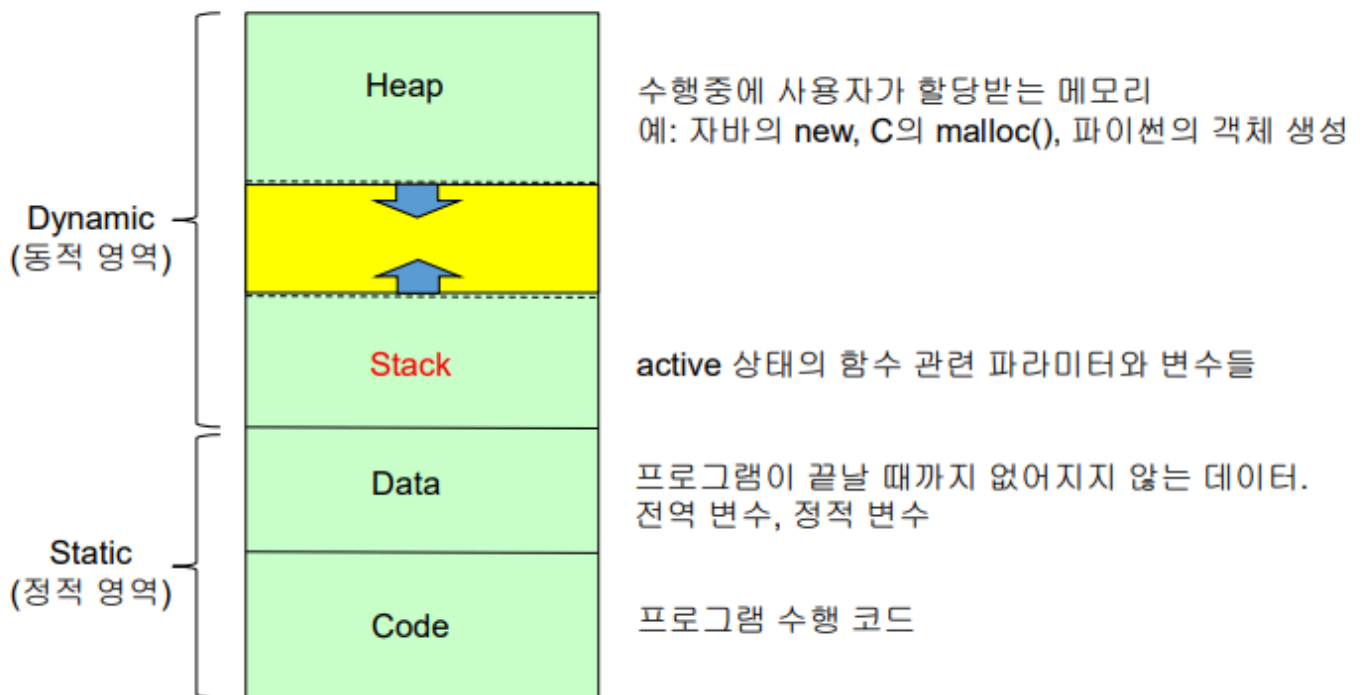


■ 스택의 개념과 원리

- 넣을 때는 위로 쌓아 올리고, 뺄 때는 위에서부터 뺀다.
- 맨 윗 원소의 내용만 볼 수 있다.

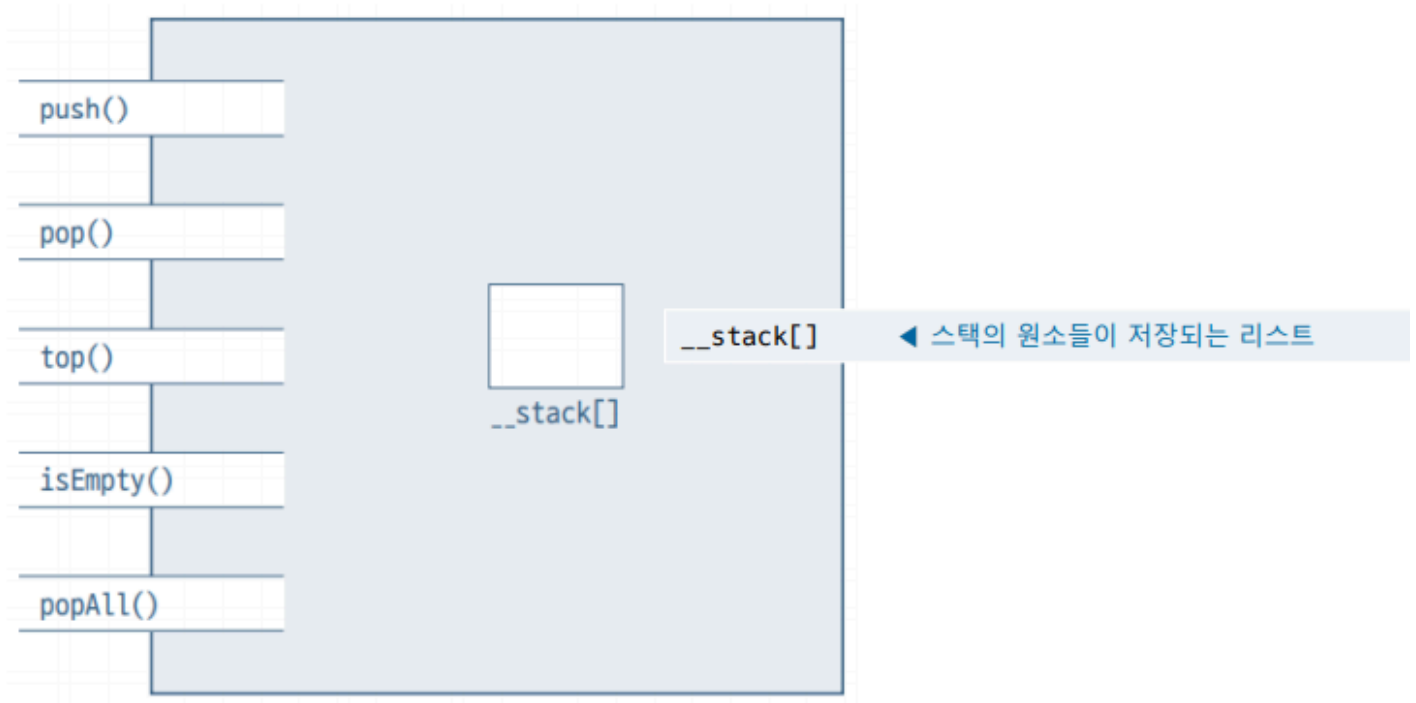


■ 프로그램 수행과 스택 활용



1. 리스트를 이용한 스택

▪ 리스트 스택의 객체 구조



push()	push(x) ◀ 스택의 맨 위 원소에 x를 삽입한다.
pop()	pop() ◀ 스택의 맨 위에 있는 원소를 알려주고 삭제한다.
top()	top() ◀ 스택의 맨 위에 있는 원소를 알려준다.
isEmpty()	isEmpty() ◀ 스택이 비어있는지 알려준다.
popAll()	popAll() ◀ 스택을 깨끗이 청소한다.

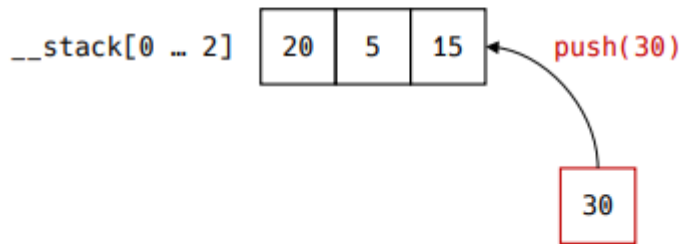
▪ 리스트 스택의 작업과 구현

▪ ListStack.py

```
class ListStack:
    def __init__(self):
        self.__stack = []
    def push(self, x): ...
    def pop(self): ...
    def top(self): ...
    def isEmpty(self): ...
    def popAll(self): ...
```

push()

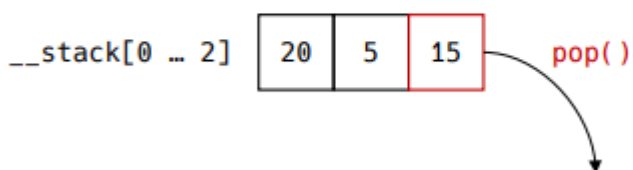
- 원소 삽입



```
def push(self, x):  
    self.__stack.append(x)
```

pop()

- 원소 삭제



```
def pop(self):  
    return self.__stack.pop()
```

Top()

- 스택의 탑 원소 알려주기

__stack[0 ... 2] 20 5 15 __stack[] []

 __stack[-1]

```
def top(self):
    if self.isEmpty():
        return None
    else:
        return self.__stack[-1]
```

IsEmpty()

- 스택이 비었는지 확인하기

```
__stack[0 .. 2]
```

20	5	15
----	---	----

```
bool(self.__stack) : True
```

```
def isEmpty(self) -> bool:
    return not bool(self.__stack)
```

popAll()

- 스택 비우기

__stack[0 ... 2]	20	5	15
------------------	----	---	----

```
def popAll(self):
    self.__stack = []
```

구현

- self.__stack.pop() 은 stack이 리스트인데 리스트의 pop이다.

■ 파이썬 구현

```
class ListStack:
    def __init__(self):
        self.__stack = []

    def push(self, x):
        self.__stack.append(x)

    def pop(self):
        return self.__stack.pop()

    def top(self):
        if self.isEmpty():
            return None
        else:
            return self.__stack[-1]

    def isEmpty(self) -> bool:
        return not bool(self.__stack)
        # 또는 return len(self.__stack) == 0

    def popAll(self):
        self.__stack.clear()

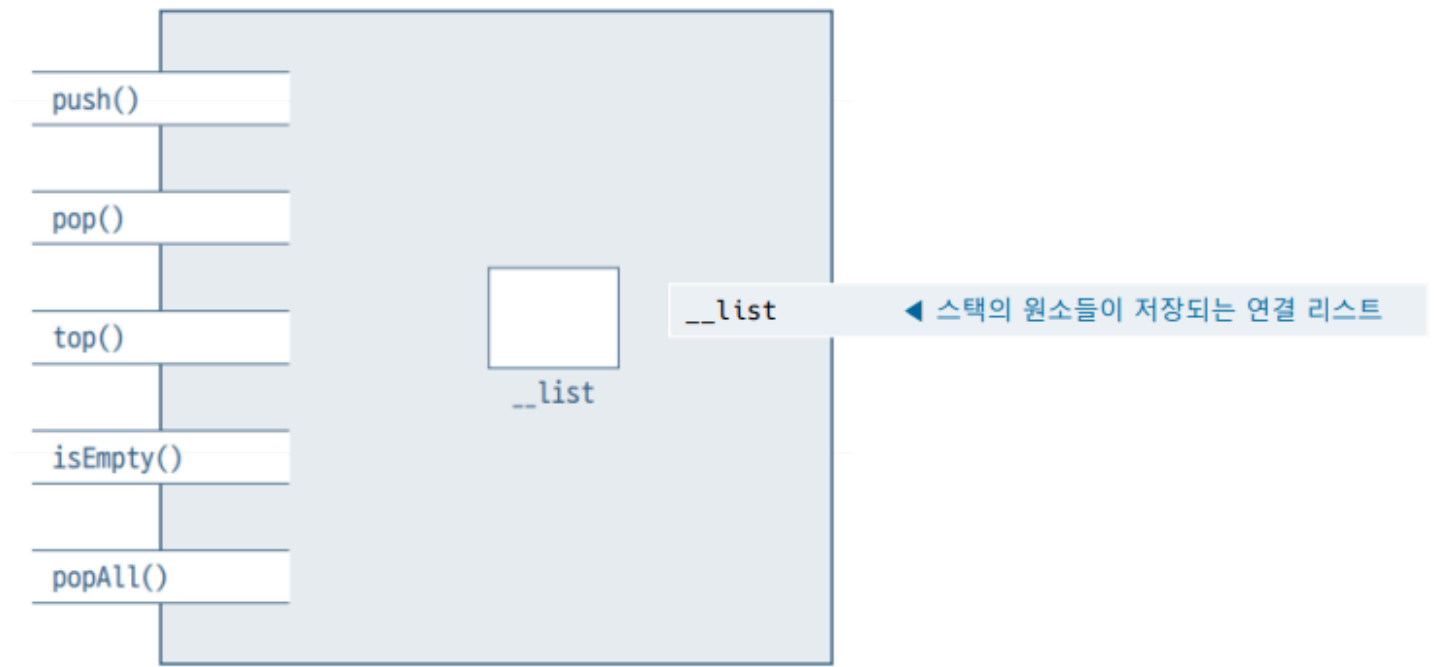
    def printStack(self):
        print("Stack from top:", end=' ')
        for i in range(len(self.__stack) - 1, -1, -1):
            print(self.__stack[i], end=' ')
        print()
```

```
st1 = ListStack()
print(st1.top()) # No effect
st1.push(100)
st1.push(200)
print("Top is", st1.top())
st1.pop()
st1.push('Monday')
st1.printStack()
print('isEmpty?', st1.isEmpty())
```

```
Top is 200
Stack from top: Monday 100
isEmpty? False
```

2. 연결 리스트를 이용한 스택

▪ 연결리스트 스택의 객체 구조



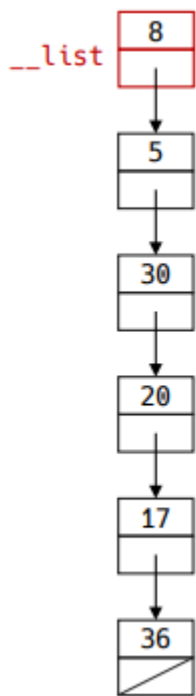
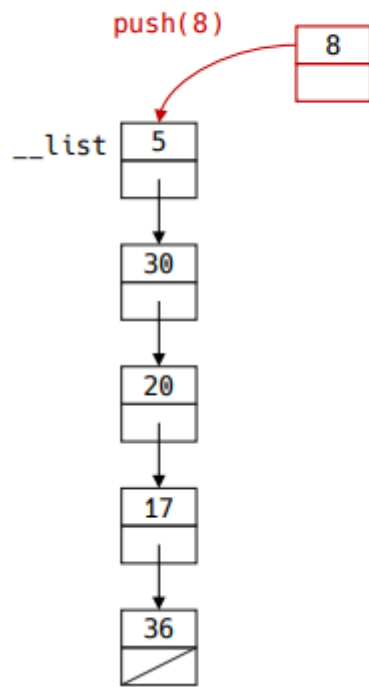
- 연결리스트 스택의 작업과 구현

- linkedStack.py

```
class LinkedStack:
    def __init__(self):
        self.__list = LinkedListBasic()
    def push(self, x): ...
    def pop(self): ...
    def top(self): ...
    def isEmpty(self): ...
    def popAll(self): ...
```

원소 삽입

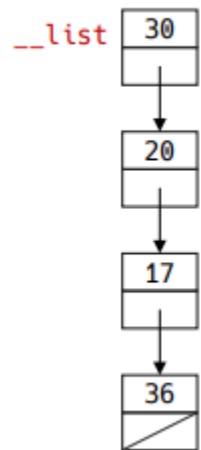
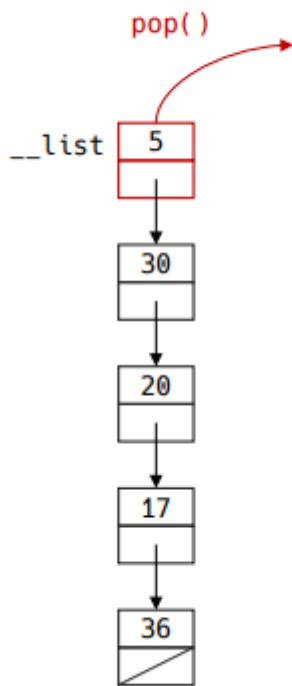
원소 삽입



```
def push(self, x):  
    self.__list.insert(0, x)
```

원소 삭제

- 원소 삭제



```
def pop(self):
    self.__list.pop(0)
```

기타 작업

- 기타 작업

```
def top(self):
    if self.isEmpty():
        return None
    else:
        return self.__list.get(0)
```

```
def isEmpty(self):
    return self.__list.isEmpty()
```

```
def popAll(self):
    self.__list.clear()
```

구현

- 파이썬 구현

```
class LinkedStack:
    def __init__(self):
        self.__list = LinkedListBasic()

    def push(self, newItem):
        self.__list.insert(0, newItem)

    def pop(self):
        return self.__list.pop(0)

    def top(self):
        return self.__list.get(0)

    def isEmpty(self) -> bool:
        return self.__list.isEmpty()

    def popAll(self):
        self.__list.clear()

    def printStack(self):
        print("Stack from top:", end=' ')
        for i in range(self.__list.size()):
            print(self.__list.get(i), end=' ')
        print()
```

```
st1 = linkedStack()
st1.push(100)
st1.push(200)
print("Top is", st1.top())
st1.pop()
st1.push('Monday')
st1.printStack()
print('isEmpty?', st1.isEmpty())
```

```
Top is 200
Stack from top: Monday 100
isEmpty? False
```

그 외까지

3. 스택 응용

문자열 뒤집기

▪ 문자열 뒤집기

- 주어진 문자열의 순서를 뒤집는 작업
- 문자열이 끝날 때까지 push()를 통해 스택에 저장
- 스택이 빌 때까지 pop()을 통해 문자를 하나씩 빼면서 출력

```
def reverse(str):
    st = ListStack()
    for i in range(len(str)):
        st.push(str[i])
    out = ""
    while not st.isEmpty():
        out += st.pop()
    return out

def main():
    input = "Test Seq 12345"
    answer = reverse(input)
    print("Input string: ", input)
    print("Reversed string: ", answer)

if __name__ == "__main__":
    main()
```

Postfix 계산

▪ Postfix 계산

```
def evaluate(p):
    s = ListStack()
    digitPreviously = False
    for i in range(len(p)):
        ch = p[i]
        if ch.isdigit():
            if digitPreviously:
                tmp = s.pop()
                tmp = 10 * tmp + (ord(ch) - ord('0'))
                s.push(tmp)
            else:
                s.push(ord(ch) - ord('0'))
                digitPreviously = True
        elif isOperator(ch): # ch가 연산자
            s.push(operation(s.pop(), s.pop(), ch))
            digitPreviously = False
        else:
            digitPreviously = False
    return s.pop()

def isOperator(ch) -> bool:
    return (ch == '+' or ch == '-'
            or ch == '*' or ch == '/')

def operation(opr2:int, opr1:int, ch) -> int:
    return {'+': opr1 + opr2, '-': opr1 - opr2,
            '*': opr1 * opr2, '/': opr1 // opr2}[ch]
```

```
def main():
    postfix = "700 3 47 + 6 * - 4 /"
    print("Input string: ", postfix)
    answer = evaluate(postfix)
    print("Answer: ", answer)
    print(ord('0'), ord('9'))

if __name__ == "__main__":
    main()
```

postfix = "700 3 47 + 6 * - 4 /"

번외 - 선택 정렬

■ 선택 정렬

- 배열 $A[0, \dots, n-1]$ 의 n 개의 원소
- 목표: n 개의 원소를 크기순으로 정렬

알고리즘 2-6 선택 정렬(재귀 버전)

`selectionSort(A[], n):` ◀ 배열 $A[0 \dots n-1]$ 을 정렬한다

① if ($n > 1$)

② $A[0 \dots \text{last}]$ 중 가장 큰 수 $A[k]$ 를 찾는다

③ $A[k] \leftrightarrow A[\text{last}]$ ◀ $A[k]$ 와 $A[\text{last}]$ 의 값을 교환한다

④ `selectionSort(A, n-1)` ◀ 배열 $A[0 \dots n-2]$ 를 정렬한다

■ 선택 정렬

- `SelectionSort(A, n)`

– 최대 원소를 찾는다.

29	10	14	37	13
----	----	----	----	----

– 최대 원소와 맨 오른쪽 자리 원소를 바꾼다.

29	10	14	37	13
----	----	----	----	----

– 맨 오른쪽 자리를 관심에서 제외한다.

29	10	14	13	37
----	----	----	----	----

– 원소가 1개 남을 때 까지 위를 반복한다.

29	10	14	13	37
----	----	----	----	----

전위 중위 후위 표기법

■ 중위, 전위, 후위 표현법

- 수식을 표현할 때 연산자의 상대적 위치에 따라
 - 전위 표현 (prefix expression)
 - 중위 표현 (infix expression)
 - 후위 표현 (postfix expression)

로 나뉜다.

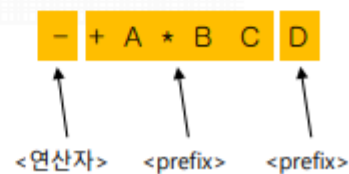
중위 표현법		전위 표현법	후위 표현법
$A + B * C - 2$	→	$- + A * B C 2$	$A B C * + 2 -$
$(A + B) * (C - 2)$	→	$* + A B - C 2$	$A B + C 2 - *$

그림 2-3 중위, 전위, 후위 표현법 예

■ 중위, 전위, 후위 표현법

(a) 중위 표현법	$\langle \text{infix} \rangle = \langle \text{변수} \rangle \mid \langle \text{infix} \rangle \langle \text{연산자} \rangle \langle \text{infix} \rangle$ $\langle \text{연산자} \rangle = + \mid - \mid * \mid /$ $\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$
(b) 전위 표현법	$\langle \text{prefix} \rangle = \langle \text{변수} \rangle \mid \langle \text{연산자} \rangle \langle \text{prefix} \rangle \langle \text{prefix} \rangle$ $\langle \text{연산자} \rangle = + \mid - \mid * \mid /$ $\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$
(c) 후위 표현법	$\langle \text{postfix} \rangle = \langle \text{변수} \rangle \mid \langle \text{postfix} \rangle \langle \text{postfix} \rangle \langle \text{연산자} \rangle$ $\langle \text{연산자} \rangle = + \mid - \mid * \mid /$ $\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

그림 2-4 재귀가 포함된 중위, 전위, 후위 표현법의 형식 언어



일반적

$3 + 2 * 5$

→
 $+ 3 * 2 5$

더한다 3과 곱한다 2와

←
 $3 2 5 * +$
더한다 (곱한다 5와 2)와 3

수학적 귀납법

<증명: 수학적 귀납법>

- i) (경계조건 만족) 우선 0개를 옮기는 데는 이동 횟수가 0번이다. $T(0) = 2^0 - 1 = 0$ 이므로 만족한다.
- ii) (귀납적 가정) k 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k) = 2^k - 1$ 이라 가정하자.
- iii) (귀납적 전개: $k+1$ 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k+1) = 2^{k+1} - 1$ 이 됨을 보인다) $k+1$ 개의 원반을 옮기는 작업은 원반 k 개를 옮기는 작업 두 번, 원반 1개를 옮기는 작업 한 번으로 구성되므로 다음이 성립한다.

$$T(k+1) = 2T(k) + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$$